

Como Fazer Requisições com a Fetch API

1. Introdução à Fetch API

1.1. Por que Fetch?

A **Fetch API** é a interface moderna e baseada em Promises do JavaScript para fazer requisições de rede. Ela substituiu o antigo XMLHttpRequest (XHR) por oferecer uma sintaxe mais limpa, mais flexível e mais fácil de integrar com o ecossistema assíncrono do JavaScript [1].

Conceito: A Fetch API é a abordagem moderna para requisições HTTP, baseada em Promises, que simplifica a comunicação com APIs e servidores.

1.2. Sintaxe Básica

A função fetch() recebe um ou dois argumentos:

- 1 **url (obrigatório):** O endereço do recurso que você deseja acessar.
- 2 **options (opcional):** Um objeto de configuração para requisições mais complexas (como POST, PUT, ou para adicionar cabeçalhos).

A função sempre retorna uma **Promise** que resolve para o objeto Response.

```
fetch(url, options)

.then(response => {
  // Processa a resposta
})
.catch(error => {
  // Trata erros de rede
});
```

2. Requisição GET (Leitura de Dados)

A requisição GET é o método padrão e é usada para buscar dados de um servidor.

2.1. Exemplo Simples

Quando você omite o objeto options, o fetch assume automaticamente o método GET.

```
// Exemplo de busca de dados de um usuário

fetch('https://jsonplaceholder.typicode.com/users/1')
  .then(response => {
    // 1. O Fetch não lança erro para status HTTP 4xx ou 5xx.
    // É crucial verificar a propriedade 'ok' da resposta.
    if (!response.ok) {
      throw new Error(`Erro HTTP! Status: ${response.status}`);
    }
    // 2. Converte o corpo da resposta para JSON
    return response.json();
  })
  .then(usuario => {
    console.log("Dados do Usuário:", usuario);
    // Saída: { id: 1, name: "Leanne Graham", ... }
  })
  .catch(error => {
    console.error("Falha na requisição:", error.message);
  });
```

2.2. Tratamento da Resposta

É fundamental entender que a Promise retornada por fetch() só será rejeitada (caindo no .catch()) em caso de **falha de rede** (ex: sem conexão, erro de DNS). Para erros HTTP (como 404 Not Found ou 500 Internal Server Error), a Promise é resolvida, mas a propriedade response.ok será false.

Propriedade	Descrição
<u>response.ok</u>	Booleano. <u>true</u> se o status HTTP for 200-299.
<u>response.status</u>	O código de status HTTP (ex: 200, 404).
<u>response.json()</u>	Método para ler o corpo da resposta como JSON. Retorna uma nova Promise.
<u>response.text()</u>	Método para ler o corpo da resposta como texto puro. Retorna uma nova Promise.

3. Requisição POST (Criação de Dados)

Requisições POST são usadas para enviar dados ao servidor, geralmente para criar um novo recurso. Elas exigem o uso do objeto options.

3.1. Configuração Essencial

Para um POST que envia dados JSON, o objeto options deve incluir:

- 3 **method: 'POST'**: Define o método HTTP.
- 4 **headers**: Um objeto que define os cabeçalhos. O mais importante é o Content-Type: 'application/json', que informa ao servidor o formato dos dados enviados.
- 5 **body**: O corpo da requisição. O objeto JavaScript deve ser convertido para uma string JSON usando JSON.stringify().

3.2. Exemplo Prático de POST

```
const novoPost = {  
  
  title: 'Meu Novo Post',  
  body: 'Este é o conteúdo do meu post.',  
  userId: 1,  
};  
  
fetch('https://jsonplaceholder.typicode.com/posts', {
```

```
method: 'POST', // 1. Define o método
headers: {
  'Content-Type': 'application/json', // 2. Informa o tipo de dado
},
body: JSON.stringify(novoPost), // 3. Converte o objeto para string JSON
})
.then(response => {
  if (!response.ok) {
    throw new Error(`Falha ao criar post. Status: ${response.status}`);
  }
  return response.json();
})
.then(data => {
  console.log("Post Criado com Sucesso:", data);
  // O servidor geralmente retorna o objeto criado, incluindo um novo ID.
})
.catch(error => {
  console.error("Erro ao enviar dados:", error.message);
});
```

4. A Abordagem Moderna: Async/Await

Embora o encadeamento de `.then()` funcione, a sintaxe `async/await` torna o código mais limpo e fácil de depurar, especialmente em operações complexas.

4.1. Exemplo de GET com Async/Await

Usamos o bloco `try...catch` para lidar com erros de rede e erros HTTP que lançamos manualmente.

```
async function buscarProduto(id) {

  const url = `https://api.exemplo.com/produtos/${id}`;

  try {
    // 1. Pausa a execução até a resposta da rede
```

```
const response = await fetch(url);

// 2. Tratamento de erro HTTP
if (!response.ok) {
  throw new Error(`Erro ${response.status}: Recurso não encontrado.`);
}

// 3. Pausa a execução até a conversão do JSON
const produto = await response.json();

return produto;

} catch (error) {
  console.error("Ocorreu um erro na busca:", error.message);
  return null; // Retorna nulo em caso de falha
}
}

// Chamada da função
buscarProduto(42).then(data => {
  if (data) {
    console.log("Produto:", data.nome);
  }
});
```

4.2. Exemplo de POST com Async/Await

```
async function criarUsuario(dadosUsuario) {

const url = 'https://api.exemplo.com/usuarios';

try {
  const response = await fetch(url, {
    method: 'POST',
    headers: {
      'Content-Type': 'application/json',
    },
    body: JSON.stringify(dadosUsuario),
  });

  if (!response.ok) {
```

```
    throw new Error(`Falha na criação. Status: ${response.status}`);
  }

  const novoUsuario = await response.json();
  console.log("Usuário criado com ID:", novoUsuario.id);
  return novoUsuario;

} catch (error) {
  console.error("Erro ao criar usuário:", error.message);
  return null;
}
}

// Chamada da função

criarUsuario({ nome: "Maria", email: "maria@exemplo.com" });
```

5. Outros Métodos HTTP

A Fetch API pode ser usada para todos os métodos HTTP, bastando alterar a propriedade method no objeto options.

Método HTTP	Propósito	Configuração Fetch
GET	Buscar dados.	Omitido (padrão)
POST	Criar um novo recurso.	<u>method: 'POST'</u>
PUT	Substituir/Atualizar um recurso inteiro.	<u>method: 'PUT'</u>
PATCH	Atualizar parcialmente um recurso.	<u>method: 'PATCH'</u>
DELETE	Remover um recurso.	<u>method: 'DELETE'</u>

Exemplo de DELETE:

```
fetch('https://api.exemplo.com/produtos/10', {
```

```
  method: 'DELETE',
```

```
})
```

```
.then(response => {
```

```
  if (response.status === 204) {
```

```
    console.log("Recurso 10 deletado com sucesso (204 No Content).");
```

```
  } else if (!response.ok) {
```

```
    throw new Error(`Falha ao deletar. Status: ${response.status}`);
```

```
  }
```

```
});
```